

Consistent branching across a container and a database

EnvBranch mini-task memo · Songhan (Mason) Wu · direction: coordinating branches across components

1. Motivation

An agent is fixing a failing integration test in a web service. Inside a Waypoint-managed environment it has cloned the repo, applied a half-finished schema migration, started the app server in the background, and holds a warm connection pool to Postgres over seeded data. It now wants three siblings: fix the migration, fix the ORM model, or roll back and re-seed.

Checkpoint, restore and fork on the container capture the agent's *belief* about the database — the ORM identity map, pooled connections, cached sequence values, an open transaction on a socket fd — but not the database itself. CRIU restores the socket descriptor; it cannot restore the peer's session: prepared statements, temp tables, session GUCs, advisory locks, open portals. The StateFork study names this as an open challenge, noting that restoring a checkpoint invalidates state *held by the remote peer* [SF25].

So a logical branch here is a **pair** (container checkpoint *C*, database branch *B*), and the question is not whether we can snapshot both, but whether the pair forms a **consistent cut**. Two independently timed snapshots do not:

- **Orphan** (*C* after *B*): the restored container believes transaction *X* committed — it received the ack, updated its caches, logged it — but *B* was cut before *X*. The agent then debugs a world that looks reproducible and is corrupt.
- **Duplicate** (*B* after *C*): *B* holds writes the container has no memory of issuing, so on restore it re-issues them — duplicate rows, unique-constraint violations, burned sequence values.

Neither is data loss. Both are *silent divergence*, which is worse: a branching agent treats restored state as ground truth, and every sibling rollout descending from a bad pair inherits the corruption.

2. Idea and related work

What the literature gives. DeltaBox [DB26] and Fork, Explore, Commit [FEC26] make the intra-sandbox branch nearly free — 14 ms checkpoint / 5 ms rollback, and sub-350 μ s branch creation — by making copy-on-write the default and never bulk-copying. Spice [SP26] repeats the lesson one layer down, reaching 0.6–18 ms restore by decoupling a snapshot's storage layout from the process's virtual-memory layout. All three optimize *one* state domain. BranchBench [BB26] measures the other and finds branchable DBMSes sitting on a hard tradeoff curve: a 5–4000 $\times$ read penalty as lineage deepens versus a 25–1500 $\times$ branch create/switch cost, with content-addressed tree traversal identified as the culprit [BB26, §5.5, p. 10]. It also explicitly scopes out the surrounding application [BB26, §2, p. 2]. Both halves are individually fast; nobody has said what it means to branch them *together*.

The asymmetry. In Chandy–Lamport every process is a peer that can only record "now", which is exactly why markers must be pushed through channels to assemble a cut [CL85, §3.2, p. 10]. Here the two components are *not* peers: the container can only be checkpointed at wall-clock now, because CRIU freezes the process tree; the database is *already versioned* and can branch at an arbitrary past point (Neon by LSN, Dolt by commit). That asymmetry removes the need for a marker protocol. Checkpoint the container at now, then **retroactively** branch the database at the last commit the container actually observed.

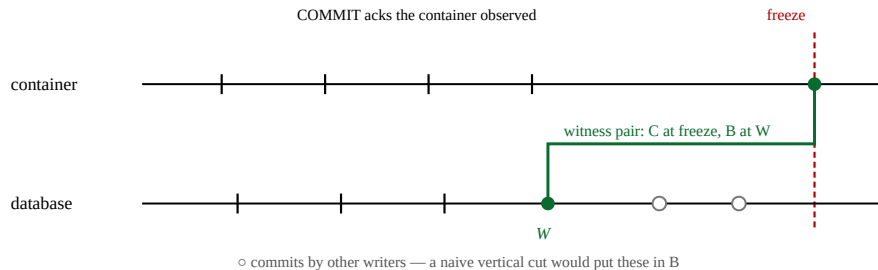


Fig. 1 — The naive vertical cut is not consistent. The witness pair hooks back to *W*, the last commit the container observed.

Smallest useful mechanism. A commit-witness shim on the container's connection path — a driver wrapper, or a loopback proxy inside the container's network namespace — keeps one value in a shared page: *W* = max, over all connections, of the witness token seen on a COMMIT ack. On Postgres that token is a WAL LSN, and the shim pipelines COMMIT with `pg_current_wal_insert_lsn()`, so the witness costs one extra statement and no extra round trip. The witness must be an *upper bound* on the observed commit's LSN and never a lower bound — an LSN read immediately after the ack satisfies this, while a wall-clock approximation has no ordering relation to LSN and can round either way, producing orphans. Rounding up is already covered by the non-guarantee below; rounding down would break the guarantee.

```
envbranch create: freeze container (Waypoint) -> C    # W lives in C, captured atomically with the freeze
                  branch database at W           -> B
                  manifest { C, (backend, B, W), session_descriptors }
```

```
envbranch restore: restore C; shim rebinds each connection to B and replays session_descriptors
                    (search_path, session GUCs, prepared statements, temp-table DDL)
```

Guarantee (one-sided, deliberately). Every transaction whose commit ack the container observed before the freeze is present in *B*: no orphans.

Explicit non-guarantee. If the database has other writers, *B* may contain commits the container never observed. A two-sided cut requires exclusive write access or a real marker protocol — which is precisely the channel-recording step this design trades away.

The limitation I would fail loudly on. If a connection is frozen inside a write transaction, the channel is non-empty and no correct pairing at *W* exists. *v1 refuses* the checkpoint rather than restoring a world in which that transaction never happened; advisory locks and open portals refuse for the same reason. Refusing is the point — silent divergence is the failure mode being removed, so a loud refusal is a better outcome than a plausible lie.

What can stay shared. Read-only reference data — seeded corpora, extension catalogs, any relation no branch writes — never needs forking; only the writable tail does. This is the same structure that makes LSN-addressed backends cheap and content-addressed ones expensive under depth [BB26, §5.5, p. 10].

3. Evaluation

Run two BranchBench agentic workloads (*failure reproduction, software engineering*) through a container-hosted application with a real connection pool and ORM cache, rather than a bare SQL client.

Arm	Definition
B0 naive	container checkpoint, then an independent database branch immediately after — today's default
B1 quiesce	drain the pool, wait for idle, then branch both — correct but pays a stall
B2 witness	the proposed retroactive pairing at <i>W</i>

Primary metric — divergence rate: the fraction of restored branches in which the application's observable state disagrees with its database branch, measured mechanically by a fixed reconciliation probe run after every restore (the app claims order *N* exists → SELECT it; compare the app-visible sequence high-water mark against the database's). **Secondary:** branch-create latency, and shim overhead per COMMIT in μ s.

Hypothesis. B0's divergence rate is non-zero and grows with write concurrency; B2 drives it to ≈ 0 at materially lower create latency than B1.

What would tell me to drop this. If B0's divergence is already ≈ 0 under realistic agent workloads — because agents write rarely and the freeze window is short relative to commit latency — the mechanism buys nothing and "just quiesce" is the right answer; a small measured B1 stall would confirm it. It also dies if the shim's per-COMMIT overhead lands on the same order as the branch-create latency it saves.

Closing

The part I would most like to own is the container–database boundary itself: pinning down what a consistent pair means, and building the shim and manifest that make a restore refuse rather than lie. EECS 482 is where I built the pieces this is made of — a virtual-memory pager with copy-on-write and file-backed shared pages, and a multithreaded network file server whose entire correctness argument was about concurrent access to shared state across descriptors.

References

- [CL85] K. M. Chandy and L. Lamport. Distributed Snapshots: Determining Global States of Distributed Systems. *ACM TOCS* 3(1):63–75, 1985. <https://lamport.azurewebsites.net/pubs/chandy.pdf> (*found independently*)
- [BB26] E. Ang, S. Weldon, I. K. Kim, K. Durand, K. Kaffes, E. Wu. BranchBench: Aligning Database Branching with Agentic Demands. arXiv:2604.17180. <https://arxiv.org/abs/2604.17180> (*found independently*)
- [DB26] Y. Dong, J. He, S. Liu, Y. Hou, D. Du, Z. Xu, S. Yu, B. Yang, Y. Xia, H. Chen. DeltaBox: Scaling Stateful AI Agents with Millisecond-Level Sandbox Checkpoint/Rollback. arXiv:2605.22781. <https://arxiv.org/abs/2605.22781>
- [FEC26] C. Wang, Y. Zheng. Fork, Explore, Commit: OS Primitives for Agentic Exploration. arXiv:2602.08199. <https://arxiv.org/abs/2602.08199>
- [SP26] B. Holmes, B. Dinis, L. Honcharuk, A. Belay, J. Fried. Rethinking Process Snapshots for Near-Warm Serverless Cold Starts. *OSDI* 2026. <https://www.usenix.org/conference/osdi26/presentation/holmes>
- [SF25] J. Xu, T. Zhou, E. Wu, K. Kaffes. Toward Systems Foundations for Agentic Exploration. arXiv:2510.05556. <https://arxiv.org/abs/2510.05556>